NVIDIA Isaac Sim i Isaac Lab: uczenie robotów, AGILE, Arena i cyfrowe bliźniaki

Raport syntetyczny · wersja 1.1.0 · 12 listopada 2026 r.
Zakres: ekosystem symulacji, uczenia i ewaluacji polityk robotycznych, w tym humanoidów, modele świata i wdrożenia przemysłowe.

Streszczenie

Ekosystem NVIDIA Isaac łączy symulację fizyczną i sensoryczną, wielkoskalowe uczenie polityk oraz ewaluację zachowania robotów w środowiskach wirtualnych. Niniejszy raport scala dwa nakładające się opracowania o Isaac Sim, Isaac Lab, AGILE i bliźniakach cyfrowych. Wyjaśnia role OpenUSD, Isaac Sim i Isaac Lab, następnie omawia uczenie ze wzmocnieniem, imitację, benchmarki Isaac Lab-Arena, generowanie danych syntetycznych oraz przejście *sim-to-real*. Oddziela elementy platformy od projektów dodatkowych i przykładów przemysłowych. Wnioskiem jest konieczność walidacji polityki zarówno w symulacji, jak i na docelowej platformie sprzętowej, z kontrolą opóźnień, bezpieczeństwa i odporności. Raport stanowi przegląd techniczny, nie sprawozdanie z własnego eksperymentu.

1. Architektura ekosystemu NVIDIA Isaac

OpenUSD (Universal Scene Description) jest formatem i strukturą reprezentacji scen 3D, obejmującą hierarchię obiektów, ich właściwości i powiązania między warstwami danych. Może wspierać integrację modeli CAD, geometrii, materiałów i metadanych. Sama zgodność z OpenUSD nie gwarantuje jednak zgodności modeli dynamiki, czujników ani semantyki interakcji.

Isaac Sim jest środowiskiem symulacyjnym wykorzystującym m.in. PhysX, renderowanie RTX, modele czujników i integracje ROS 2. Służy do projektowania scen, symulacji ruchu i kontaktów oraz tworzenia danych syntetycznych. **Isaac Lab** to otwarty framework uczenia robotów, rozwijany w powiązaniu z Isaac Sim i innymi backendami, obsługujący uczenie ze wzmocnieniem (RL), uczenie przez imitację (IL), symulację równoległą oraz zarządzanie zadaniami [1, 2]. Wcześniejsze opracowania opisywały Isaac Lab wyłącznie jako warstwę Isaac Sim; nowsza architektura 3.x umożliwia również wybrane przepływy bez uruchamiania pełnego Isaac Sim [3].

Komponent	Zakres odpowiedzialności	Typowe zastosowanie
OpenUSD	Reprezentacja i kompozycja scen;	Cyfrowe modele zakładów,

Komponent	Zakres odpowiedzialności	Typowe zastosowanie
	wymiana wybranych danych 3D	geometria, metadane
Isaac Sim	Symulacja fizyki, czujników i renderowania; integracje robotyczne	Testy percepcji, generowanie danych, integracja ROS 2
Isaac Lab	Konfiguracja zadań i środowisk, trening i ewaluacja polityk	RL, IL, równoległe eksperymenty, randomizacja domeny
Isaac Lab-Arena	Modułowe tworzenie i wykonywanie benchmarków	Powtarzalna ocena polityk i generalizacji
AGILE / WBC-AGILE	Projekt zewnętrzny na Isaac Lab, ukierunkowany na sterowanie humanoidami	Trening lokomocji i manipulacji całym ciałem
NVIDIA Cosmos	Modele świata i narzędzia generowania/transformacji danych	Wzbogacanie danych i ocena scenariuszy

Równoległe środowiska umożliwiają próbkowanie wielu epizodów na GPU. Dane obserwacji, akcji i nagród przechowuje się zwykle w strukturach tensorowych, ograniczając koszt przesyłania między procesorem i kartą graficzną. Korzyści zależą jednak od złożoności sceny, modeli czujników, liczby kontaktów i ograniczeń pamięci. Integracja z ROS 2 jest realizowana przez odpowiednie mosty i pakiety; nie jest to odpowiednik jednego uniwersalnego protokołu komunikacyjnego.

1.1. Tryby tworzenia aplikacji Isaac Sim

Starszy raport `isaac_sim.docx` opisuje dwa przepływy pracy: rozszerzenia i autonomiczne skrypty Python. Dokumentacja Isaac Sim 6.2 wyróżnia **trzy** sposoby pracy:

Tryb	Charakterystyka	Zastosowania i uwagi
GUI	Interaktywna	Projektowanie i

Tryb	Charakterystyka	Zastosowania i uwagi
	edycja scen USD, robotów, sensorów i grafów OmniGraph	inspekcja scen, konfiguracja narzędzi
Extension	Rozszerzenia Omniverse Kit; interakcja asynchroniczna, obsługa interfejsu, opcjonalny hot reload	Narzędzia programisty, wizualizacja, prototypy
Standalone Python	Kontrola kroków symulacji i renderowania, również bez GUI	Automatyzowane eksperymenty, trening RL, reprodukowalna generacja danych

W trybie Extension logika aplikacyjna współistnieje z pętlą Kit; operacje blokujące i sterowanie czasem wymagają odpowiedniego planowania callbacków oraz funkcji asynchronicznych. W Standalone Python krok fizyki można wykonywać jawnie, po wykonaniu operacji aktualizujących scenę. Samo używanie plików USD nie rozstrzyga o trybie uruchomienia. Por. Isaac Sim 6.2: Development Methods.

1.2. Core API i migracja do nowszych wersji

W źródłach `isaac_sim.docx` oraz `isaac_sim_pojecia.pdf` wymieniono m.in. `World`, `Scene`, `SimulationContext`, `Articulation` i `DynamicCuboid`. Są to **pojęcia historycznego interfejsu programistycznego**, a nie uniwersalne, niezależne od wersji gwarancje dostępności klas. Począwszy od Isaac Sim 6.0 rozszerzenia `isaacsim.core.api`, `isaacsim.core.prims` i `isaacsim.core.utils` oznaczono jako przestarzałe.

Starszy element lub pojęcie	Odpowiednik i znaczenie przy migracji
`World`, `SimulationContext`	Zarządzanie cyklem symulacji przez `SimulationManager` i narzędzia `isaacsim.core.experimental.utils.stage`; nie migrować wywołań mechanicznie 1:1
`Scene` i historyczne pomocnicze obiekty Core API	Reprezentacja sceny USD i nowsze interfejsy zarządzania

Starszy element lub pojęcie	Odpowiednik i znaczenie przy migracji
	prymitywami
`Articulation` z `isaacsim.core.prims`	Nowsze `isaacsim.core.experimental.prims`, z weryfikacją ścieżek importu i kontraktu
Obiekty geometrii, np. `DynamicCuboid`	`isaacsim.core.experimental.objects` lub bezpośrednie API OpenUSD; zgodność parametrów wymaga testu
Callbacki kroków fizyki	Interfejsy cyklu życia `SimulationManager`; konfiguracja w odniesieniu do wersji runtime

W repozytorium eksperymentu należy zapisać wersję Isaac Sim oraz testować używane symbole/API. Starsze przykłady oparte na `World.step()` mogą zachować wartość dydaktyczną, ale dla nowego kodu nie powinny być proponowane bez ostrzeżenia migracyjnego. Źródło: NVIDIA, Core API to Core Experimental API.

1.3. Reprezentacja robota, fizyka i import modeli

Dodatkowy materiał `isaac_sim.docx` opisuje import modeli **URDF**, **MJCF** oraz złożeń CAD, konfigurację przegubów i inspekcję ich ograniczeń. Należy wyraźnie oddzielić trzy warstwy: (i) geometrię i strukturę kinematyczną, (ii) parametry dynamiki oraz kontaktu, (iii) sterownik i jego założenia dotyczące częstotliwości oraz opóźnień. Import modelu nie potwierdza poprawności mas, bezwładności, charakterystyk napędów i kontaktu. Należy zweryfikować mapowanie połączeń i limitów stawów, konwencje współrzędnych, jednostki oraz zachowanie modelu przy znanych trajektoriach testowych.

Opisany w oryginale Onshape Importer jest przykładem **specjalizowanej integracji CAD**, której dostępność i zakres funkcji należy sprawdzać dla wybranej wersji oraz konfiguracji. Podobnie `cuOpt` dotyczy optymalizacji tras i harmonogramowania (np. VRP), a nie domyślnej funkcji dynamiki robota. Przy projektowaniu magazynu sensowne jest oddzielenie grafu tras, planowania floty i sterowania lokalnego pojedynczej platformy.

1.4. Dane sensorów, metadata i układy odniesienia

Starsze materiały szczegółowo opisują `GenericModelOutput` (GMO) jako nośnik danych i metadanych sensorów RTX. Współczesna dokumentacja Isaac Sim wskazuje, że m.in. wynik sensora RTX Lidar może trafiać do `GenericModelOutput` AOV, z którego dalsze komponenty pozyskują próbki i

pola pomocnicze. W zależności od konfiguracji mogą występować identyfikatory obiektów, intensywność, normalne, dane o punktach oraz informacje wymagane do publikowania wiadomości ROS 2. **Nie należy zakładać, że wszystkie modalności mają identyczny układ pamięci i komplet tych samych pól.**

Dla powtarzalności eksperymentu trzeba zapisać orientację i układ osi sensora, ramę odniesienia `frame_id`, przekształcenia do układu robota, znaczniki czasu, okres próbkowania, parametry szumów i kompensację ruchu, jeśli jest używana. Przykładowe osie sensora (+x w przód, +y w lewo, +z w górę) z dokumentu źródłowego należy sprawdzać **w odniesieniu do konkretnej dokumentacji sensora**, a nie przenosić jako regułę dla każdego źródła danych.

Przykład wersjozależności: w dokumentacji RTX Sensors odnotowano problem pola `echoId` w RTX Lidar dla wersji Isaac Sim 6.0 i 6.1; pole może mieć wartość zero nawet przy wielokrotnym odbiciu. Dlatego walidacja strumienia powinna obejmować nie tylko liczbę punktów i rozkład odległości, lecz także sensowność metadanych. Zob. RTX Sensors oraz RTX Lidar.

1.5. ROS 2, DDS i synchronizacja

Integracja z ROS 2 obejmuje publikowanie obserwacji (np. obrazy, chmury punktów, odometrię i transformacje), odbieranie poleceń oraz sterowanie czasem symulacji. Oryginalny dokument wymienia `On Physics Step` w OmniGraph jako mechanizm wiązania obliczeń sterownika z krokami fizyki. Należy jawnie sprawdzić semantykę czasu symulowanego, opóźnienia i częstotliwości publikacji, szczególnie gdy policy inference działa wolniej niż pętla dynamiki.

DDS (Data Distribution Service) jest mechanizmem komunikacji, na którym opierają się typowe implementacje ROS 2. Podstawowe terminy z oryginalnego słownika:

Pojęcie	Znaczenie
`DomainParticipant`	Uczestnik określonej domeny DDS, działający w granicach przypisanej konfiguracji
`Topic`	Nazwany i typowany kanał danych
`DataWriter`	Publikacja próbek o zdefiniowanym typie
`DataReader`	Odbiór próbek z tematu
IDL	Definicja typów i kontraktów danych, a nie transport

Pojęcie	Znaczenie
	wiadomości

Powyższe nazwy odnoszą się do modelu DDS, a nie są specyficznym interfejsem Isaac Sim. Wywołania `read()`, `take()`, `write()` i `dispose()` należy wiązać z konkretną implementacją biblioteki, np. Eclipse Cyclone DDS, oraz jej wersją. W Isaac Sim 6.2 dokumentowane i testowane są konfiguracje ROS 2 Humble oraz Jazzy; uruchomienie innych dystrybucji i wersji Pythona wymaga osobnej kontroli kompatybilności (ROS 2 installation).

1.6. Replicator i syntetyczne dane uczące

Wcześniejszy raport opisuje narzędzia **Semantics Schema Editor**, **Synthetic Data Recorder** i **Replicator**. Warstwa adnotacji semantycznych umożliwia przypisanie znaczenia obiektom, a adnotatory mogą generować np. RGB, maski segmentacji lub prostokąty detekcji. Randomizacja oświetlenia, tekstur, położenia obiektów i kamer dostarcza zmienności scen, ale nie dowodzi automatycznie zgodności z rozkładem danych rzeczywistych.

`BasicWriter` jest podstawowym przykładem zapisu danych syntetycznych, a `CustomWriter` umożliwia wyspecjalizowany eksport. Należy rozróżnić interfejs **Synthetic Data Recorder** od konfiguracji innych rozszerzeń. Przykładowo od wersji 1.6.1 `isaacsim.replicator.agent.core` nie akceptuje `BasicWriter` jako klucza najwyższego poziomu w swojej konfiguracji i wymaga odpowiedniej deklaracji `CustomWriter`; nie znaczy to, że `BasicWriter` całkowicie usunięto z Replicatora. Źródła: Synthetic Data Recorder, Configuration File Guide.

Zalecane metadane eksportu: wersja Isaac Sim i odpowiednich rozszerzeń, hash sceny USD i modeli robota, seed randomizacji, parametry sensorów i renderera, konfiguracja generatora, schemat etykiet, liczba próbek oraz osobny protokół oceny danych rzeczywistych.

Pochodzenie uzupełnienia: dodatkowe fragmenty pochodzą z `files/inne/analizy/isaac_sim.docx` (SHA-256: `1044199a58993861648083af45c6fff759e4781f6281dd353aff4c4ec42be5a3`) oraz `isaac_sim_pojecia.pdf` (SHA-256: `883d25e545ea3b92c5ba0a8c401bdbd63902828a04da858cfc812598d9da4909`). Uaktualnienie pojęć i kompatybilności zweryfikowano z dokumentacją Isaac Sim 6.2; zawartość źródeł nie była automatycznie uznawana za aktualną.

2. Uczenie polityk i projekt AGILE

AGILE (*A Generic Isaac-Lab based Engine for humanoid loco-manipulation learning*) jest osobnym projektem WBC-AGILE zbudowanym na Isaac Lab, a **nie silnikiem fizycznym Isaac Sim**. Projekt udostępnia przepływ od konfiguracji zadania i treningu po ewaluację i transfer na roboty fizyczne, z

przykładami dla platform humanoidalnych, w tym Unitree G1 i Booster T1 [4].

Proces badawczy warto podzielić na pięć kroków:

1. **Kontrola modelu robota.** Zweryfikować geometrię, ograniczenia stawów, masę i bezwładność, punkty kontaktu, jednostki oraz zgodność aktuatorów z robotem docelowym.
2. **Projekt zadania.** Zdefiniować obserwacje, nagrody, zdarzenia kończące epizod, warunki początkowe, ograniczenia bezpieczeństwa i sposób logowania.
3. **Trening i dobór hiperparametrów.** Rejestrować konfigurację, wersje kodu, losowe ziarna, krzywe uczenia, przebiegi wideo i parametry polityki; rozdzielać trening od oceny.
4. **Ewaluacja ilościowa.** Raportować błędy trajektorii, częstość upadków, skuteczność osiągania celu, odporność na zakłócenia i wyniki dla scenariuszy niewidzianych w treningu.
5. **Transfer i wdrożenie.** Przetestować interfejsy akcji i obserwacji, częstotliwości sterowania, opóźnienia, ograniczenia momentu oraz procedury zatrzymania awaryjnego; potwierdzić zgodność wersji runtime.

Nie należy traktować pojedynczej demonstracji humanoida ani wyniku w jednym symulatorze jako wystarczającego dowodu niezawodności polityki. WBC-AGILE dokumentuje również techniki destylacji *teacher–student* i metody oceny, które pomagają odróżnić trening przy obserwacjach uprzywilejowanych od wdrożenia na rzeczywistych sensorach [4].

3. Isaac Lab-Arena i afordancje

Isaac Lab-Arena służy do budowania benchmarków, wykonywania ewaluacji na wielu scenach i identyfikowania warunków, w których polityka zawodzi [5]. Poziom ewaluacji należy odróżniać od poziomu samego treningu. Powtarzalne benchmarki powinny raportować wersje środowiska, rozkłady inicjalizacji, losowe ziarna i metryki, a także informacje o nieudanych epizodach.

Afordancja (*affordance*) jest opisem możliwej interakcji między robotem a obiektem, np. **Openable** (otwieranie), **Pressable** (naciskanie) lub interakcja z pokrętłem. API afordancji umożliwia definiowanie zadania na poziomie funkcji obiektu, bez wiązania całej logiki z konkretną siatką 3D. Przykładowo polecenie otwarcia można zastosować do różnych typów pojemników o kompatybilnym interfejsie [6]. Jednak wspólna afordancja nie usuwa różnic geometrii, sił kontaktu i ograniczeń mechanicznych; ich obsługa pozostaje problemem kontroli i percepcji.

4. Dane syntetyczne i modele świata

NVIDIA Cosmos stanowi rodzinę narzędzi i modeli świata wykorzystywanych do generowania lub modyfikowania danych reprezentujących sceny dynamiczne. Materiały źródłowe rozróżniały m.in. mechanizmy typu **Transfer**, służące zmianie wybranych cech wizualnych, i **Predict**, związane z przewidywaniem lub generowaniem przyszłych stanów sceny. Takie narzędzia mogą wspierać randomizację domeny, różnicowanie wyglądu i tworzenie zbiorów do treningu modeli percepcji lub modeli vision–language–action (VLA).

Dane syntetyczne nie zastępują automatycznie danych rzeczywistych. Ich wartość zależy od podobieństwa rozkładów oświetlenia, dynamiki, geometrii, zakłóceń i zachowania sensora do warunków wdrożenia. Szczególnie istotna jest ocena *domain gap*: syntetyczne nagrania mogą utrwalać błędy symulatora. Dlatego eksperyment powinien uwzględniać zbiory z rzeczywistych sensorów i przypadki brzegowe, a nie opierać się wyłącznie na liczbie wygenerowanych epizodów.

5. Real-to-sim, sim-to-real i uczenie przez imitację

Real-to-sim obejmuje pozyskiwanie parametrów i trajektorii z obiektu rzeczywistego oraz przygotowanie odpowiedniego modelu wirtualnego. Mogą tu wystąpić niezgodności w geometrii, kalibracji czujników, tarciu, opóźnieniach i charakterystykach aktuatorów. **Sim-to-real** oznacza przeniesienie polityki sterowania wyuczonej lub dostrojonej w symulacji do robota, który działa przy ograniczeniach rzeczywistego sprzętu.

Typowe ryzyka to niedokładne modelowanie kontaktu, błędne limity stawów, niedoszacowane opóźnienia komunikacji, rozbieżne częstotliwości aktualizacji polityki i symulatora oraz niespójność definicji obserwacji. Stosuje się randomizację parametrów symulacji, identyfikację układu, walidację w trybie *hardware-in-the-loop* i bezpieczny, etapowy wzrost autonomii. Dla humanoidów szczególne znaczenie ma interakcja stóp z podłożem, ale nie można zakładać uniwersalnej wartości błędu transferu (np. w centymetrach) bez konkretnego protokołu testowego.

Imitation learning wykorzystuje demonstracje człowieka albo innego sterownika. W teleoperacji konfiguracja *leader–follower* pozwala gromadzić pary obserwacja–akcja. Istotnym problemem jest akumulacja błędów i przesunięcie rozkładu stanów (*covariate shift*) podczas autonomicznego działania. Pomocne mogą być dodatkowe demonstracje, korekty online lub uzupełnianie treningu interakcją z symulatorem. Materiały źródłowe wymieniały tanie ramiona klasy SO-100 jako przykład sprzętu do pozyskiwania demonstracji; konkretny protokół i kompatybilność sprzętowa wymagają osobnej dokumentacji.

6. Cyfrowe bliźniaki przemysłowe

Cyfrowy bliźniak nie jest synonimem estetycznej wizualizacji. Powinien zachowywać odniesienie do obiektu rzeczywistego, reprezentować istotne parametry i umożliwiać aktualizację danych oraz analizę scenariuszy. OpenUSD i Omniverse mogą stanowić warstwę kompozycji i wizualizacji, lecz odpowiedzialność za wymianę danych produkcyjnych, aktualność modelu i walidację zależy od rozwiązania integracyjnego.

W obu dokumentach źródłowych opisano przykłady **BMW** (planowanie przestrzeni fabrycznych), **Pegatron** (integracja danych CAD i robotów przemysłowych) oraz **SN Twin** (tworzenie modeli z istniejących danych firmowych). Przykłady te pokazują kierunki zastosowań, nie są natomiast dowodem powtarzalnego efektu liczbowego w każdym zakładzie. W szczególności wzmianka o 99,8% skuteczności inspekcji w pierwotnym raporcie nie zawierała niezależnie udokumentowanego zbioru walidacyjnego, więc nie powinna być cytowana jako ogólny parametr dokładności rozwiązania.

7. Proponowany protokół eksperymentalny

Dla porównywalnego badania polityki należy zapisać: (i) wersje Isaac Lab/Isaac Sim i silnika fizycznego, (ii) model i parametry robota, (iii) inicjalizację i liczbę środowisk, (iv) obserwacje, akcje, funkcję nagrody, (v) seeds i liczbę niezależnych treningów, (vi) metryki sukcesu i bezpieczeństwa, (vii) testy na scenach niewidzianych, (viii) stopień różnicy między symulacją a fizyczną platformą, (ix) sposób kontroli awaryjnej oraz (x) dostępność kodu i danych. Dla systemów VLA i generatywnych modeli świata należy osobno badać wrażliwość na błędne obserwacje oraz niepewność predykcji.

Wnioski

Isaac Sim służy przede wszystkim jako środowisko symulacji i generowania danych, Isaac Lab organizuje trening i testowanie polityk, Isaac Lab-Arena dostarcza warstwę benchmarków, a WBC-AGILE ilustruje specjalistyczny przepływ uczenia sterowania humanoidami. OpenUSD integruje reprezentację scen, a Cosmos może wspierać przygotowanie danych syntetycznych. Zastosowanie tych narzędzi w jednej architekturze nie likwiduje jednak potrzeby weryfikacji realnego sprzętu, ryzyka kontaktu, ograniczeń czujników i kontroli poprawności modelu świata.

Bibliografia i dokumentacja techniczna

1. Mittal, M. i in. (2025). *Isaac Lab: A GPU-Accelerated Simulation Framework for Multi-Modal Robot Learning*. NVIDIA Research. https://research.nvidia.com/publication/2025-09_isaac-lab-gpu-accelerated-simulation-framework-multi-modal-robot-learning

2. NVIDIA Developer. *Isaac Lab*. https://developer.nvidia.com/isaac/lab
3. Isaac Lab, repozytorium i informacje o wersjach 3.x. https://github.com/isaac-sim/IsaacLab/releases
4. NVIDIA Isaac / WBC-AGILE. *A Generic Isaac-Lab based Engine for humanoid loco-manipulation learning*. https://github.com/nvidia-isaac/WBC-AGILE
5. NVIDIA Developer. *Isaac Lab-Arena*. https://developer.nvidia.com/isaac/lab-arena
6. Isaac Lab-Arena. *Affordances*. https://isaac-sim.github.io/IsaacLab-Arena/release/0.3.0/pages/concepts/scene/concept_affordances_design.html
7. NVIDIA. *Isaac Sim – ROS 2*. https://docs.isaacsim.omniverse.nvidia.com/latest/ros2_tutorials/index.html

Pochodzenie i zakres weryfikacji

Raport syntetyzuje `Isaac_Lab.docx`, `Isaac Lab i bliźniaki cyfrowe.pdf`, `isaac_sim.docx` i `isaac_sim_pojecia.pdf` z `files/inne/analizy/`, z aktualizacją terminologii na podstawie oficjalnych źródeł. Zawartość tabeli porównawczej pochodzi z materiałów pierwotnych, lecz została skorygowana o rozwój architektury Isaac Lab. Pominięto niesprawdzone ogólne twierdzenia o dokładności AOI i arbitralnej precyzji sterowania; nie są prezentowane jako wynik badań własnych. Historyczne pliki nie są nadpisywane.